

P3787R2

Adjoints to "Enabling list-initialization for algorithms": uninitialized_fill

Published Proposal, 2026-03-24

Authors:

[Giuseppe D'Angelo](#)

[Ruslan Arutyunyan](#)

Audience:

LWG, LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We extend the same changes that [\[P2248R8\]](#) applied to the rest of the find algorithms to `std<::ranges>::uninitialized_fill`.

Table of Contents

1 Changelog

2 Motivation and scope

3 Proposed Wording

4 Acknowledgements

References

Informative References

§ 1. Changelog

- R2
 - Fixed the wording after LWG review.
- R1
 - Amended and rebased the wording in order to include the parallel range algorithms.
- R0
 - First submission

§ 2. Motivation and scope

In the Tokyo 2024 meeting [\[P2248R8\]](#) (Enabling list-initialization for algorithms) was adopted.

Due to an oversight, the `std::uninitialized_fill` family of algorithms was excluded from the ones where a defaulted template parameter was added.

We propose to modify `uninitialized_fill`'s specification, so that it matches the post-P2248 one for the rest of the algorithms (especially `fill`).

§ 3. Proposed Wording

All the proposed changes are relative to [\[N5032\]](#).

In `[version.syn]`, bump the feature-test macro as shown:

```
#define __cpp_lib_algorithm_default_value_type 202403L-YYYYMML
// also in <algorithm>, <ranges>, <string>, <deque>, <list>, <forward_list>
```

where the new value corresponds to the year and month of adoption of the present proposal.

NOTE: [\[P3217R0\]](#) was also submitted as a similar fix for `find_last`. That proposal did **not** bump the feature-test macro, as it was considered a "hotfix". However, the time-frame is different for the present proposal: there are already implementations shipping with [\[P2248R8\]](#), and therefore we consider it safer to bump the value again.



Modify [memory.syn] and the corresponding signatures in the entries under [uninitialized.fill] as shown:

```
template<class NoThrowForwardIterator, class T = iterator_traits<NoThrowForwardIterator>>
    constexpr void uninitialized_fill(NoThrowForwardIterator first,
                                     NoThrowForwardIterator last, const T& x);

template<class ExecutionPolicy, class NoThrowForwardIterator, class T = iterator_traits<NoThrowForwardIterator>>
    void uninitialized_fill(ExecutionPolicy&& exec,
                           NoThrowForwardIterator first,
                           NoThrowForwardIterator last,
                           const T& x);

template<class NoThrowForwardIterator, class Size, class T = iterator_traits<NoThrowForwardIterator>>
    constexpr NoThrowForwardIterator
        uninitialized_fill_n(NoThrowForwardIterator first, Size n, const T& x);

template<class ExecutionPolicy, class NoThrowForwardIterator, class Size, class T = iterator_traits<NoThrowForwardIterator>>
    NoThrowForwardIterator
        uninitialized_fill_n(ExecutionPolicy&& exec,
                              NoThrowForwardIterator first,
                              Size n, const T& x);

namespace ranges {
    template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S, class T = iter_value_t<I>>
        requires constructible_from<iter_value_t<I>, const T&>
        constexpr I uninitialized_fill(I first, S last, const T& x);
    template<nothrow-forward-range R, class T = range_value_t<R>>
        requires constructible_from<range_value_t<R>, const T&>
        constexpr borrowed_iterator_t<R> uninitialized_fill(R&& r, const T& x);

    template<nothrow-forward-iterator I, class T = iter_value_t<I>>
        requires constructible_from<iter_value_t<I>, const T&>
        constexpr I uninitialized_fill_n(I first,
                                         iter_difference_t<I> n, const T& x);

    template<execution-policy Ep, nothrow-random-access-iterator I,
            nothrow-sized-sentinel-for<I> S, class T = iter_value_t<I>>
        requires constructible_from<iter_value_t<I>, const T&>
        I uninitialized_fill(Ep&& exec, I first, S last, const T& x);

    template<execution-policy Ep, nothrow-sized-random-access-range R, class T = range_value_t<R>>
        requires constructible_from<range_value_t<R>, const T&>
```

```

        borrowed_iterator_t<R> uninitialized_fill(Ep&& exec, R&& r,
                                                const T& x);

template<execution_policy Ep, nothrow-random-access-iterator I, class T>
requires constructible_from<iter_value_t<I>, const T&>
    I uninitialized_fill_n(Ep&& exec, I first,
                          iter_difference_t<I> n, const T& x);
}

```

§ 4. Acknowledgements

Thanks to Ruslan Arutyunyan for spotting this oversight.

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[N5032]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 15 December 2025.

URL: <https://wg21.link/n5032>

[P2248R8]

Giuseppe D'Angelo. *Enabling list-initialization for algorithms*. 20 March 2024. URL:

<https://wg21.link/p2248r8>

[P3217R0]

Giuseppe D'Angelo. *Adjoints to "Enabling list-initialization for algorithms": find_last*. 20240331.

URL: <https://wg21.link/p3217r0>